DineEase — Full-Stack Restaurant Management & Reservation System

1. Introduction

1.1 Purpose

DineEase is an end-to-end technical solution designed for restaurant owners and customers. It streamlines the dining experience by providing a digital interface for seat reservations, menu browsing, and user authentication.

1.2 Target Audience

- **Customers:** Users looking to discover restaurants and book tables.
- **Restaurant Staff/Admins:** Personnel managing table availability and user records.
- **Developers:** Those interested in MERN stack architecture and secure authentication flows.

1.3 Learning Outcomes

- Secure **JWT-based** Authentication.
- Password encryption using **Bcryptjs**.
- State management in **React** (Hooks & Context).
- Handling asynchronous API calls with **Axios**.
- Responsive UI design with modern CSS.

---

2. System Overview

2.1 User Roles

Role	Description
Guest	Can browse the home page and view the restaurant mission.
Customer	Can register, log in, and make table reservations.
Admin	Can view all registered users and manage reservation schedules.

2.2 Core Features

- **User Authentication:** Secure registration and login.
- **Real-time Feedback:** Integrated react-hot-toast for success/error alerts.
- **Session Management:** Persistent login using HTTP-only cookies.
- **Auto-Redirect:** Success page with countdown timer post-registration.

---

3. High-Level Architecture

DineEase follows the **MERN** (MongoDB, Express, React, Node) architecture pattern:

- **Frontend:** React.js (Single Page Application).
- **Backend:** Node.js & Express.js (RESTful API).
- **Database:** MongoDB (Cloud Atlas).
- **Security:** JWT for authorization and Bcrypt for data hashing.

  MERN Flow -

  React (5173)

  ↓ CORS

  Express (4000)

  ↓ Mongoose

  MongoDB Atlas

---

4. Database Design

User Credentials and Reservation Details

JSON

```
{
  "_id": "ObjectId",
  "name": "string",
  "email": "string (unique)",
  "password": "string (hashed)",
  "role": "string (default: 'customer')",
  "createdAt": "date"
}
```

5. Backend API Design

Backend/

```
|── config/
|   └── config.env
|── database/
|   └── dbConnection.js
```

```
|— node_modules/
|— app.js
|— server.js
|— package.json
```

app.js → Express configuration

server.js → Server start + DB connection

database/ → MongoDB logic

config.env → Environment variables

6. Frontend Architecture

```
src/
 ├— components/     # Reusable UI (Navbar, ProtectedRoute)
 ├— Pages/          # Home, Register, Success
 ├— App.jsx         # Main Router & Toast provider
 ├— App.css         # Global styling
 └— main.jsx        # Entry point
```

7. Security Considerations

- **HTTP-Only Cookies:** Prevents Cross-Site Scripting (XSS) by hiding tokens from JavaScript.
- **Password Salting:** Using bcrypt.hash() with a salt factor of 10.
- **Route Guarding:** Frontend ProtectedRoute component to prevent unauthorized access to the Success page.
- **Input Validation:** Using validator library on the backend to ensure legitimate email formats.

8. Development Workflow

1. **Backend Setup:** Configure MongoDB connection and Error Middleware.
2. **Schema Design:** Build the User model with pre-save hashing logic.
3. **API Development:** Create registration and login controllers.
4. **Frontend Integration:** Connect React forms to Express endpoints via Axios.
5. **Testing:** Verify token persistence and redirect logic.

9. Conclusion

DineEase is a full-stack **MERN (MongoDB, Express.js, React.js, Node.js)** based restaurant web application designed to simplify restaurant operations and enhance the customer dining experience.

The platform enables users to explore restaurants, view menus, place orders, and manage reservations through a modern and user-friendly interface